DorPatch: Distributed Occlusion-Robust Adversarial Patch Attack Against Certified Defenses

1. Introduction

In recent years, Deep Learning models have been widely used in security-critical applications such as autonomous vehicles, surveillance systems, facial recognition, medical diagnosis, and intelligent traffic systems. Although these models achieve high accuracy, they are vulnerable to adversarial attacks.

An adversarial attack is a technique in which carefully crafted perturbations are added to input images to fool a neural network into making incorrect predictions.

Among various adversarial attacks, adversarial patch attacks are considered highly practical because the attacker only modifies a small region of the image instead of changing all pixels. These patches can be physically printed and attached to real-world objects.

Existing patch attacks such as LaVAN, LOAP, RP2, and IAP can successfully fool image classification models, but many modern defenses such as PatchCleanser can detect or neutralize these attacks by masking suspicious image regions.

To overcome these defenses, the DorPatch (Distributed Occlusion-Robust Patch) framework was proposed. DorPatch introduces a distributed and occlusion-robust adversarial patch that can evade certified defenses like PatchCleanser.

The main idea behind DorPatch is:

- Distribute the patch over multiple regions.
- Make the patch robust against masking occlusions.
- Improve perceptual quality and stealthiness.
- Maintain attack effectiveness even under defenses.

This project focuses on understanding and implementing the DorPatch adversarial attack framework using deep learning and adversarial optimization techniques.

2. Objective

The main objective of this project is to implement and analyze the DorPatch adversarial attack framework capable of bypassing certified adversarial defenses.

The project aims to:

- Generate distributed adversarial patches for image classification models.
- Make adversarial patches robust against masking defenses.
- Evaluate attack performance against PatchCleanser.
- Improve patch stealthiness and perceptual quality.
- Analyze attack robustness under physical-world conditions.
- Extend DorPatch using frequency-domain optimization with DCT.

3. Dataset Used

The project uses standard image classification datasets commonly used in adversarial machine learning research.

Dataset used:

3.1 ImageNet

ImageNet is a large-scale image classification dataset containing:

- More than 1 million images
- 1000 object classes
- High-resolution real-world images

ImageNet is used for evaluating physical-world adversarial patch performance.

4. Methodology

4.1 Victim Model Selection

A pre-trained convolutional neural network is selected as the victim classification model.

Models used:

- ResNet18
- ResNet50
- DenseNet

The model takes an image as input and predicts its class label.

The adversarial patch optimization process attempts to fool this model into producing incorrect predictions.

4.2 Distributed Adversarial Patch Generation

Traditional adversarial patches usually occupy a single square region.

DorPatch distributes perturbations across multiple image regions.

Advantages:

- Harder to detect
- Harder to remove using masking defenses
- More robust against occlusion

A binary mask is used to define where the adversarial perturbations are applied.

The patch mask is optimized during training.

4.3 Adversarial Optimization

The adversarial patch is optimized using gradient-based optimization.

The objective is:

- Fool the classifier
- Maintain visual stealthiness
- Survive masking defenses

The optimization process updates patch values iteratively using backpropagation.

Loss functions used:

● Adversarial Loss
● Density Regularization
● Group Lasso Loss
● Structural Loss

4.4 Density Regularization

Density regularization ensures that perturbations are distributed evenly across the image.

Without this regularization:

● perturbations may cluster in one location
● PatchCleanser can easily remove them

Density regularization improves:

● spatial distribution
● masking robustness
● defense evasion capability

4.5 Image Dropout for Occlusion Robustness

Image dropout is one of the main innovations of DorPatch.

During training:

● random image regions are masked
● parts of the patch become invisible
● optimization continues despite missing regions

This simulates PatchCleanser-style masking.

As a result:

● the attack becomes robust to partial removal
● the classifier remains fooled even after masking

4.6 Group Lasso Regularization

Group Lasso regularization encourages neighboring perturbations to form connected groups.

Benefits:

- improves physical realizability
- creates printable patch regions
- avoids isolated noisy pixels

This helps generate more practical adversarial patches.

4.7 Structural Loss

Structural loss improves patch perceptual quality.

It encourages:

- smooth textures
- natural image blending
- continuous perturbation structures

This reduces visible noise and makes patches less suspicious.

5. Model Pipeline

The complete project pipeline is:

Dataset Collection
↓
Data Preprocessing
↓
Victim Model Loading
↓
Distributed Mask Generation
↓
Frequency-Domain Patch Initialization
↓
Inverse DCT Patch Reconstruction
↓
Patch Application on Images
↓
Random Occlusion / Image Dropout
↓
CNN Classification

↓
Loss Computation
↓
Backpropagation
↓
Frequency Coefficient Optimization
↓
Final Evaluation

6. Data Preprocessing

The following preprocessing steps are applied before training:

- Image resizing
- Pixel normalization
- Tensor conversion
- Batch generation
- Data augmentation

7. Attack Evaluation Metrics

The following metrics are used to evaluate attack performance.

7.1 Attack Success Rate (ASR)

Measures the percentage of images successfully misclassified by the adversarial patch.

Higher ASR indicates a stronger attack.

7.2 Robust Accuracy

Measures classification accuracy under adversarial attacks.

Lower robust accuracy indicates a stronger attack.

7.3 Certified Rate of Patched Examples (CRPE)

CRPE measures:

- percentage of adversarial examples
- that are both misclassified
- and certified as safe by PatchCleanser

Higher CRPE indicates better defense evasion.

7.4 PSNR (Peak Signal-to-Noise Ratio)

Measures visual similarity between original and patched images.

Higher PSNR indicates better visual quality.

7.5 SSIM (Structural Similarity Index)

Measures structural similarity between images.

Higher SSIM indicates more natural-looking perturbations.

8. Experimental Results

Observations

- DorPatch successfully bypasses masking-based defenses.
- Distributed perturbations improve robustness.
- Image dropout significantly improves occlusion robustness.
- DCT optimization generates smoother and less visible patches.
- Frequency-domain perturbations survive JPEG compression better.
- Physical-world robustness improves under blur and resizing.

Example Expected Results

Patch #1 — Full Analysis

Stage 0 (iterations 0–1204)

Phase 1: Untargeted attack (iter 0–500)

Accuracy opens at 99.22% — the clean model is correctly classifying almost everything. By iteration 20 it collapses to 0.78%, meaning the patch has already learned to break classification across nearly all masked views. This is extremely fast convergence and is normal — the untargeted objective is easy because the patch just needs to push predictions anywhere away from the true label.

Density drops from 75.98 → 0.03% by iteration 360, and failures hit 0 at iter 100 then occasionally bounce back. This is the group lasso working — it's aggressively shrinking the mask, compressing the patch into fewer, sparser pixels while maintaining adversarialness.

Phase 2: Switch to targeted attack (iter 500, category 702)

This is the most confusing part of the log and it's completely intentional. Here's what's happening:

Before iter 500, accuracy means: "% of masked views where model still predicts the TRUE class" → you want this near 0 (attack working).

After iter 500, the target is now class 702. accuracy is recomputed as: "% of masked views where model predicts the TARGET class (702)" → you now want this near 100% because that means the patch is successfully forcing the model to predict 702 on every occluded view.

So 100% accuracy post-switch is not a failure — it is literally the attack succeeding at its hardest objective: certified targeted misclassification. Every single masked view of the patched image predicts the wrong class. That is DorPatch's core claim.

Failures also hit 0 by iteration 800, confirming PatchCleanser is fully bypassed. Early stop at 1204

Stage 1 (iterations 0–602)

Stage 1 starts fresh with the mask frozen from Stage 0. Now only the pixel values (pattern) are optimized. At iteration 0, accuracy is 1.56% — this is because the pattern has been reset to the

merged adv_x from Stage 0 end, and the new targeted optimization hasn't started yet. The pattern re-learns the targeted push extremely quickly (by iter 40) because the mask location is already optimal from Stage 0 — the hard geometry is solved, only the texture needs refinement.

The structural loss decays from 0.03 → 0.02 throughout, meaning the patch is becoming smoother and less visually detectable while remaining adversarial. Early stop at 602

9. Novel Work

Frequency-Domain DorPatch using DCT

- DorPatch is extended using DCT-based frequency optimization.
- Frequency coefficients are optimized instead of direct pixel values.
- Inverse DCT reconstructs smooth adversarial patches.
- Low-frequency perturbations improve stealthiness and JPEG robustness.
- The method improves physical-world stability and reduces visible noise.

DorPatch on transformer based architecture

- DorPatch is evaluated on newer deep learning architectures without modifying the original algorithm.
- Vision Transformer (ViT) processes images as patch sequences using self-attention.
- ViT differs from CNNs as it does not use traditional convolution operations.
- The objective is to test whether DorPatch can fool attention-based models.
- This analysis helps evaluate the transferability of adversarial attacks across architectures.

10. Advantages of DorPatch

- Effective against certified defenses
- Robust to masking occlusions
- Distributed perturbations are harder to remove
- High attack success rate
- Better physical-world realizability
- Improved perceptual quality
- Stealthier attacks compared to traditional patches

11. Advantages of Novel work

● DorPatch is extended using DCT-based frequency-domain optimization for smoother adversarial patches.

● Frequency coefficients are optimized instead of direct pixel values using inverse DCT reconstruction.

● Low-frequency perturbations improve stealthiness, JPEG robustness, and physical-world stability.

● DorPatch is also evaluated on modern architectures such as Vision Transformer (ViT).

● ViT uses self-attention and patch-based image processing instead of convolution operations.

● The objective is to analyze the transferability of adversarial patches across different neural network architectures.

12. Limitations

● Optimization process is computationally expensive.
● Physical-world evaluation requires additional setup.
● Certified defenses may evolve further.
● DCT optimization may slightly reduce attack strength.
● Large-scale datasets require GPU resources.

13. Future Work

Possible future improvements include:

● Real-time adversarial patch generation
● Video-based adversarial attacks
● Adaptive attacks against transformer defenses
● Federated adversarial learning
● Hybrid spatial-frequency optimization
● Advanced physical-world testing
● Lightweight mobile deployment

14. Technologies Used

Component	Technology
Programming Language	Python
Deep Learning Framework	PyTorch
Dataset	ImageNet
Victim Model	ResNet50
Optimization	Adam Optimizer
Visualization	Matplotlib
Hardware	GPU Recommended

15. Conclusion

This project studies and implements the DorPatch adversarial attack framework designed to evade certified defenses such as PatchCleanser.

DorPatch improves adversarial robustness by distributing perturbations across multiple image regions and introducing occlusion robustness using image dropout.

The proposed frequency-domain extension using DCT further improves:

- patch smoothness
- stealthiness
- compression robustness
- physical-world stability

The project demonstrates how adversarial machine learning can challenge modern AI security systems and highlights the importance of building stronger and more adaptive defenses.

16. References

- DorPatch Research Paper
- PatchCleanser Research Paper
- ImageNet Dataset
- PyTorch Documentation
- Adversarial Machine Learning Research Papers
- DCT and Image Compression Literature

17. Output and screenshots

```
import os
os.makedirs('/content/drive/MyDrive/DorPatch/pretrained_models/imagenet', exist_ok=True)
print("Done!")
```

```
Done!
```

```
%cd /content/DorPatch

base = "https://raw.githubusercontent.com/CGCL-codes/DorPatch/main/"
files = ["main.py", "attack.py", "utils.py", "requirements.txt", "LICENSE", "README.md"]

import urllib.request
for f in files:
    urllib.request.urlretrieve(base + f, f)
    print(f"Downloaded: {f}")
```

```
/content/DorPatch
Downloaded: main.py
Downloaded: attack.py
Downloaded: utils.py
Downloaded: requirements.txt
Downloaded: LICENSE
Downloaded: README.md
```

```
import os
os.makedirs('/content/DorPatch/defenses', exist_ok=True)
urllib.request.urlretrieve(
    "https://raw.githubusercontent.com/CGCL-codes/DorPatch/main/defenses/PatchCleanser.py",
    "/content/DorPatch/defenses/PatchCleanser.py"
)
print("Downloaded: defenses/PatchCleanser.py")
```

```
import os
os.makedirs('/content/DorPatch/defenses', exist_ok=True)
urllib.request.urlretrieve(
    "https://raw.githubusercontent.com/CGCL-codes/DorPatch/main/defenses/PatchCleanser.py",
    "/content/DorPatch/defenses/PatchCleanser.py"
)
print("Downloaded: defenses/PatchCleanser.py")
```

```
Downloaded: defenses/PatchCleanser.py
```

```
import os

# Create the directory the code expects
os.makedirs('/content/DorPatch/pretrained_models/imagenet', exist_ok=True)

# Create symlink pointing to the actual file
!ln -sf "/content/drive/MyDrive/DorPatch/pretrained_models/imagenet/checkpoints/resnetv2_50x1_bit_distilled_cutout2_128_imagenet.pth" \
        "/content/DorPatch/pretrained_models/imagenet/resnetv2_50x1_bit_distilled_cutout2_128_imagenet.pth"

# Verify it worked
!ls -la /content/DorPatch/pretrained_models/imagenet/
```

```
total 12
drwxr-xr-x 2 root root 4096 May  9 08:24 .
drwxr-xr-x 3 root root 4096 May  9 08:24 ..
lrwxrwxrwx 1 root root  123 May  9 08:24 resnetv2_50x1_bit_distilled_cutout2_128_imagenet.pth -> /content/drive/MyDrive/DorPatch/pretrained_mod
```

```
!kaggle datasets download -d ifigotin/imagenetmini-1000 \
    -p /content/data/imagenet/ --unzip
```

```
Dataset URL: https://www.kaggle.com/datasets/ifigotin/imagenetmini-1000
License(s): unknown
Downloading imagenetmini-1000.zip to /content/data/imagenet
100% 3.92G/3.92G [00:50<00:00, 83.8MB/s]
```

```
import os
val_path = '/content/data/imagenet/imagenet-mini/val'
folders = os.listdir(val_path)
print(f"Number of classes: {len(folders)}")
print(f"First 5 folders: {folders[:5]}")

# Count total images
total = sum(len(os.listdir(f'{val_path}/{f}')) for f in folders)
print(f"Total images: {total}")
```

Show hidden output

```python
from PIL import Image
import random

val_path = '/content/data/imagenet/imagenet-mini/val'
folders = os.listdir(val_path)

fig, axes = plt.subplots(2, 5, figsize=(16, 6))
axes = axes.flatten()

for i, ax in enumerate(axes):
    # pick a random class folder and random image from it
    class_folder = random.choice(folders)
    img_dir = f'{val_path}/{class_folder}'
    img_file = random.choice(os.listdir(img_dir))
    img_path = f'{img_dir}/{img_file}'

    img = Image.open(img_path).convert('RGB')
    ax.imshow(img)
    ax.set_title(class_folder, fontsize=8)
    ax.axis('off')

plt.suptitle('Random ImageNet-Mini Samples', fontsize=14)
plt.tight_layout()
plt.show()
```


```
%cd /content/DorPatch
!python main.py --data_dir /content/data/imagenet/imagenet-mini
```

```
/content/DorPatch
num_patch=-1_patch_budget=0.12
/usr/local/lib/python3.12/dist-packages/timm/models/_factory.py:138: UserWarning: Mapping deprecated model name resnetv2_50x1_bit_d
  model = create_fn(
Dataset has 3923 instances
mask size: 27, window size: 59, stride: 33
mask size: 38, window size: 69, stride: 32
mask size: 54, window size: 82, stride: 29
mask size: 77, window size: 101, stride: 25
0it [00:00, ?it/s]mask size: 27, window size: 59, stride: 33
mask size: 38, window size: 69, stride: 32
mask size: 54, window size: 82, stride: 29
mask size: 77, window size: 101, stride: 25
============ Stage 0 ============
>> 2507 failures collected!
iteration:    0, accuracy: 99.22, loss: 3.86, adv: 3.50, l2 norm: 4.00, structural: 0.06, group lasso: 28995.43, density: 75.98
iteration:   20, accuracy: 0.78, loss: 0.32, adv: 0.00, l2 norm: 4.00, structural: 0.04, group lasso: 28905.09, density: 26.70
iteration:   40, accuracy: 0.00, loss: 0.32, adv: 0.00, l2 norm: 4.00, structural: 0.02, group lasso: 28890.16, density: 28.27
iteration:   60, accuracy: 0.00, loss: 0.32, adv: 0.00, l2 norm: 4.00, structural: 0.02, group lasso: 28907.20, density: 25.76
iteration:   80, accuracy: 1.56, loss: 0.31, adv: 0.00, l2 norm: 4.00, structural: 0.01, group lasso: 28924.18, density: 18.35
>> 0 failures collected!
iteration:  100, accuracy: 0.00, loss: 0.30, adv: 0.00, l2 norm: 4.00, structural: 0.01, group lasso: 28939.31, density: 14.86
iteration:  120, accuracy: 0.78, loss: 0.31, adv: 0.00, l2 norm: 4.00, structural: 0.01, group lasso: 28908.91, density: 22.15
iteration:  140, accuracy: 0.78, loss: 0.31, adv: 0.00, l2 norm: 4.00, structural: 0.01, group lasso: 28933.93, density: 20.48
```